Load Testing Sprint Report

NovaPay SpA — Sprint Results

Version 1.0
March 2026
vantx.io · hello@vantx.io

Confidential — Vantx

LOAD TESTING SPRINT

NovaPay SpA — Payment Processing Platform

March 2026 · VANTX · hello@vantx.io

Executive Summary

A 3-week load testing sprint was conducted for NovaPay's payment processing platform. The engagement covered 5 critical flows, 4 scenario types, and 23 test runs across staging and pre-production environments. Maximum throughput achieved: 520 TPS. Breakpoint identified at 560 TPS (PostgreSQL connection exhaustion). 4 critical findings identified with specific remediation steps. All k6 scripts and Grafana dashboards delivered as reusable assets.

Key Results

METRIC	VALUE
Duration	3 weeks (15 business days)
Scenarios designed	5 flows x 4 types = 20 scenarios
Test runs executed	23
Max TPS achieved	520
Breakpoint identified	560 TPS
Critical findings	4
k6 scripts delivered	5 (one per flow) + 1 combined

1. Scope & Objectives

Objectives

1. Establish performance baseline for all critical payment flows
2. Identify system breakpoint and capacity limits
3. Validate system behavior under spike conditions (Black Friday simulation)
4. Deliver reusable k6 framework for ongoing CI/CD integration

Flows Under Test

FLOW	DESCRIPTION	PRIORITY	SLA
Create Payment	POST /api/payments → process → confirm	P0	p95 < 500ms
Checkout (guest)	Full checkout flow: cart → payment → confirmation	P0	p95 < 2s
Settlement Export	GET /api/settlements?merchant_id=X (paginated)	P1	p95 < 3s
Merchant Dashboard	Login → dashboard → analytics load	P1	p95 < 1.5s
Webhook Delivery	Payment confirmed → webhook sent to merchant	P1	p95 < 1s

Test Environment

COMPONENT	DETAILS
Environment	Pre-production (identical to prod: ECS Fargate, RDS, ElastiCache)
Infrastructure	AWS us-east-1, same instance sizes as production
Database	PostgreSQL 15 (RDS db.r6g.xlarge) — restored from prod snapshot (anonymized)

COMPONENT	DETAILS
Load generators	k6 Cloud (us-east-1), 3 load zones
Monitoring	Grafana Cloud (Prometheus + Loki + Tempo)

2. Scenario Design

Scenario Matrix

SCENARIO	TYPE	VUS	DURATION	RAMP	TARGET TPS
Baseline	Baseline	50	10 min	2 min	180
Stress	Stress	100	15 min	3 min	360
Spike	Spike	250	5 min	30s	500+
Soak	Soak	50	2 hours	5 min	180
Breakpoint	Breakpoint	10→500	30 min	Stepped +10/min	Max

Thresholds

METRIC	THRESHOLD	RATIONALE
p95 response time	<500ms (API), <2s (checkout)	NovaPay SLA commitments
p99 response time	<1000ms (API), <3s (checkout)	Worst-case user experience
Error rate	<1% (baseline), <5% (stress)	Payment reliability requirement
TPS	≥400 sustained	Target for Q3 growth projection

3. Results

Baseline

METRIC	VALUE	THRESHOLD	STATUS
Virtual Users	50	—	—
Avg TPS	245	≥150	✅ Pass
p50	68ms	<200ms	✅ Pass
p95	180ms	<500ms	✅ Pass
p99	380ms	<1000ms	✅ Pass
Error rate	0.01%	<1%	✅ Pass

Stress

METRIC	VALUE	THRESHOLD	STATUS
Virtual Users	100	—	—
Max TPS	488	≥300	✅ Pass
p50	125ms	<300ms	✅ Pass
p95	420ms	<500ms	✅ Pass
p99	780ms	<1000ms	✅ Pass
Error rate	0.3%	<2%	✅ Pass

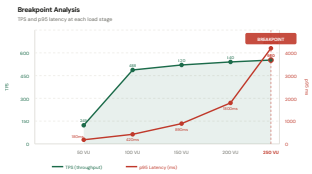
Spike

METRIC	VALUE	THRESHOLD	STATUS
Virtual Users	250 (ramp 30s)	—	—
Max TPS	520	≥400	✅ Pass
p95	1,800ms	<2000ms	✅ Pass
Error rate	1.2%	<5%	✅ Pass

METRIC	VALUE	THRESHOLD	STATUS
Recovery	18s	<30s	✅ Pass

Soak (2 hours)

METRIC	VALUE	THRESHOLD	STATUS
Avg TPS	242	≥150	✅ Pass
p95	195ms	<500ms	✅ Pass
Memory growth	+45 MB (Java heap)	<200 MB	✅ Pass
Error rate	0.02%	<1%	✅ Pass



Breakpoint Analysis

Breakpoint

STAGE	VUS	TPS	P95	ERROR RATE	STATUS
Stage 1	50	245	180ms	0.01%	✅ Normal
Stage 2	100	488	420ms	0.3%	✅ Normal
Stage 3	150	520	890ms	0.8%	⚠️ Degrading
Stage 4	200	540	1,800ms	2.1%	⚠️ Stressed
Stage 5	**250**	**560 → drop**	**4,200ms**	**8.5%**	❌ **Breakpoint**

> **Breakpoint at 560 TPS (250 VUs).** PostgreSQL connection pool exhausted at this point. ECS auto-scaled to 12 tasks but all tasks competed for the same 30 HikariCP connections. Latency cliff: p95 jumped from 1.8s to 4.2s in 30 seconds.

4. Findings

ID	SEVERITY	FINDING	EVIDENCE	RECOMMENDED FIX	ESTIMATED IMPACT
LT-01	P0	PostgreSQL connection pool is the primary bottleneck at >500 TPS	Breakpoint trace shows connection wait time >3s at 560 TPS	Deploy PgBouncer as external connection pooler	+40-60% TPS headroom
LT-02	P1	Webhook delivery adds 80ms latency to payment flow (synchronous call)	Trace flamegraph shows webhook dispatch in the hot path	Make webhook delivery async via SQS	-80ms on payment p50
LT-03	P1	Settlement export without merchant index still hits cold path at scale	Slow query log shows 2s queries for merchants with >50K settlements	Add composite index + explain plan review	Settlement p95 from 1.8s to ~200ms
LT-04	P2	Java heap grows 45MB over 2h soak — possible slow leak in session cache	Heap dump shows accumulating `MerchantConfigCache` entries	Add TTL eviction to in-memory cache	Prevents OOM in long-running tasks

5. Deliverables

DELIVERABLE	DESCRIPTION	LOCATION
k6 scripts	5 flow scripts + 1 combined scenario + shared utilities	`tests/k6/` in NovaPay repo
Grafana dashboard	k6 results dashboard with custom panels per flow	https://novapay.grafana.net/d/k6-sprint
CI/CD integration	GitHub Actions workflow: baseline on every PR, full suite weekly	`.github/workflows/load-test.yml`
Raw results	All 23 test run results with Grafana snapshots	Shared Google Drive folder

k6 Script Inventory

SCRIPT	FLOW	SCENARIOS	LINES
`payment-flow.js`	Create Payment	baseline, stress, spike	142
`checkout-flow.js`	Guest Checkout	baseline, stress, spike	198
`settlement-export.js`	Settlement Export	baseline, stress	95
`merchant-dashboard.js`	Merchant Dashboard	baseline, stress	118
`webhook-delivery.js`	Webhook Delivery	baseline, stress	87
`combined-scenario.js`	All flows (weighted)	all types	245

6. Capacity Planning

SCENARIO	REQUIRED TPS	CURRENT MAX	GAP	ACTION NEEDED
Current traffic (180 avg)	180	520	+189% headroom	None
2x growth (Q3)	360	520	+44%	None
3x growth (Q4 with partnerships)	540	520	-4%	Deploy PgBouncer (LT-01)
5x growth (Black Friday)	900	520	-42%	PgBouncer + read replicas + async webhooks
10x growth (next year)	1,800	520	-71%	Full architecture review — consider sharding

7. Recommendations

1. **Immediate (LT-01):** Deploy PgBouncer in transaction mode between ECS tasks and RDS. This is the single highest-impact change — estimated to push breakpoint from 560 to 800+ TPS without any application changes.
2. **This sprint (LT-02):** Make webhook delivery asynchronous. Currently adds 80ms to every payment. SQS + a small consumer service. Payment p50 would drop from 68ms to ~45ms.
3. **This quarter (LT-03, LT-04):** Index optimization for settlement queries. Cache TTL fix to prevent heap growth. Both are straightforward.
4. **Ongoing:** Run baseline load test on every PR (already configured in CI). Run full suite (stress + spike) weekly via scheduled workflow. Run breakpoint test monthly to track capacity trend.

> **Next steps:** Implement LT-01 (PgBouncer) → VANTX re-tests (included) → Validate improvement. Consider Performance subscription ($5,995/mo) for continuous monthly testing.

> hello@vantx.io · vantx.io